Operating Systems (OS)
Inter Process Communication (IPC) II
Process Cooperation

Prof. Dr.-Ing. Jörg Abke

TH Aschaffenburg
university of applied sciences

Copyright Hinweise

Inter-Process Communication Intro

TH Aschaffenburg
university of applied sciences

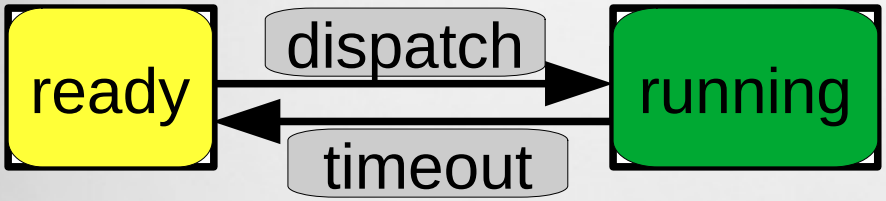
Processes

- **Undertake read, write operations on data**
- **Call each other (e.g. forking)**
- **Wait for each other**

Interaction by processes – Inter-Process Communication (IPC)

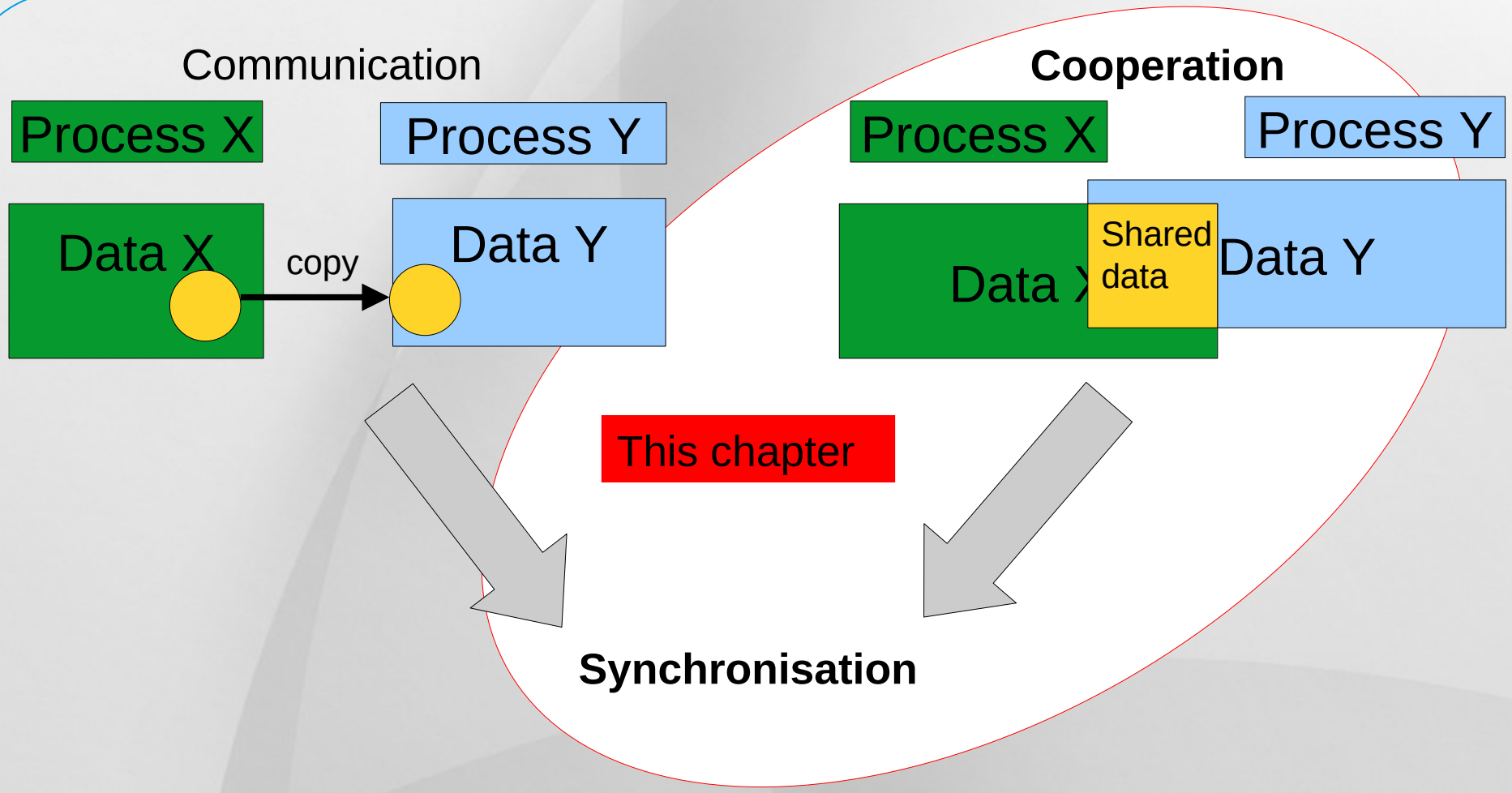
→ **following questions:**

- How to transmit/exchange data
- How to access shared ressources



Similar questions for threads

Communication, Cooperation



Cooperation

TH Aschaffenburg
university of applied sciences

- **Processes cooperation in order to controll access share data** and ressources

- Realization by
  - Unix System V

  or
  - **Posix System**

- **Ensure overlapping of critical section**
  - signal – wait (see before)
  - lock – unlock (see before)
  - **Semaphore**
  - **Mutex**
  - **Monitor**

Semaphore

TH Aschaffenburg
university of applied sciences

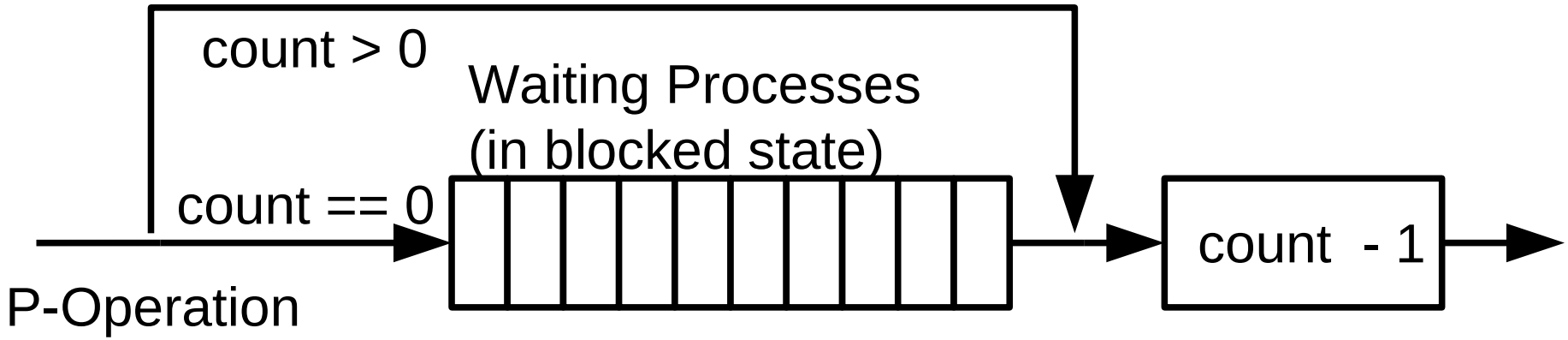
- Invented by Edsger Wye Dijkstra, Dutch Computer Scientist, in 1965
- Counting lock **S**, **Semaphore**, with operations P(S) and V(S)
  - **V** (dutch verhogen) – Raise / incraese
  - **P** (dutch proberen) - try to reduce / decrease

  - **P** and **V** are atomic operations - indivisible
    → must not be preempted or interrupted

- Advantage: More than 1 process is able to enter a critical section with countable semahores at a time

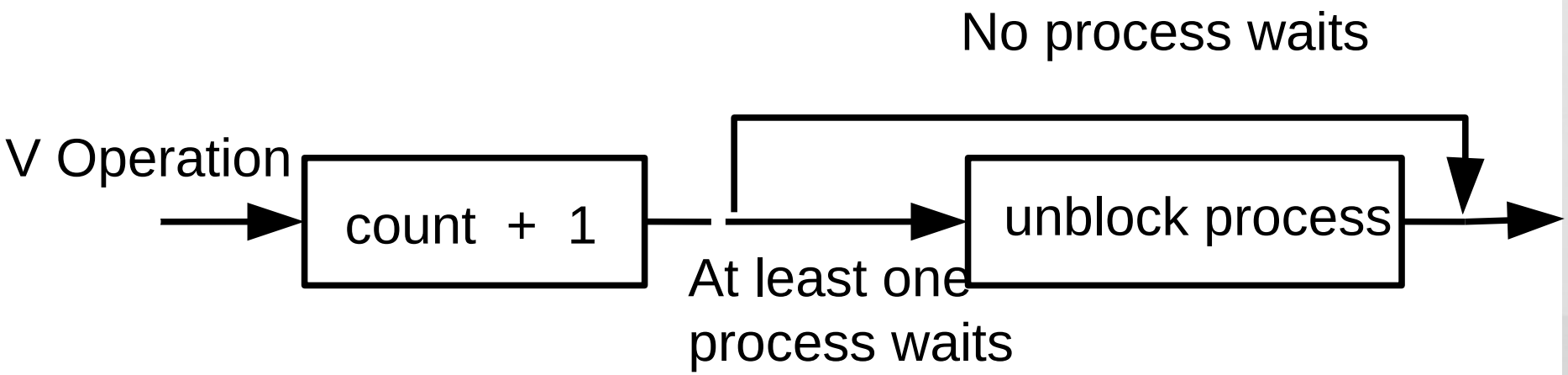
P Operation - Reduce

- **Checks whether value of the semphore is**
  - 0 : the respective process gets blocked
  - >0 : the semaphore is decremented by 1

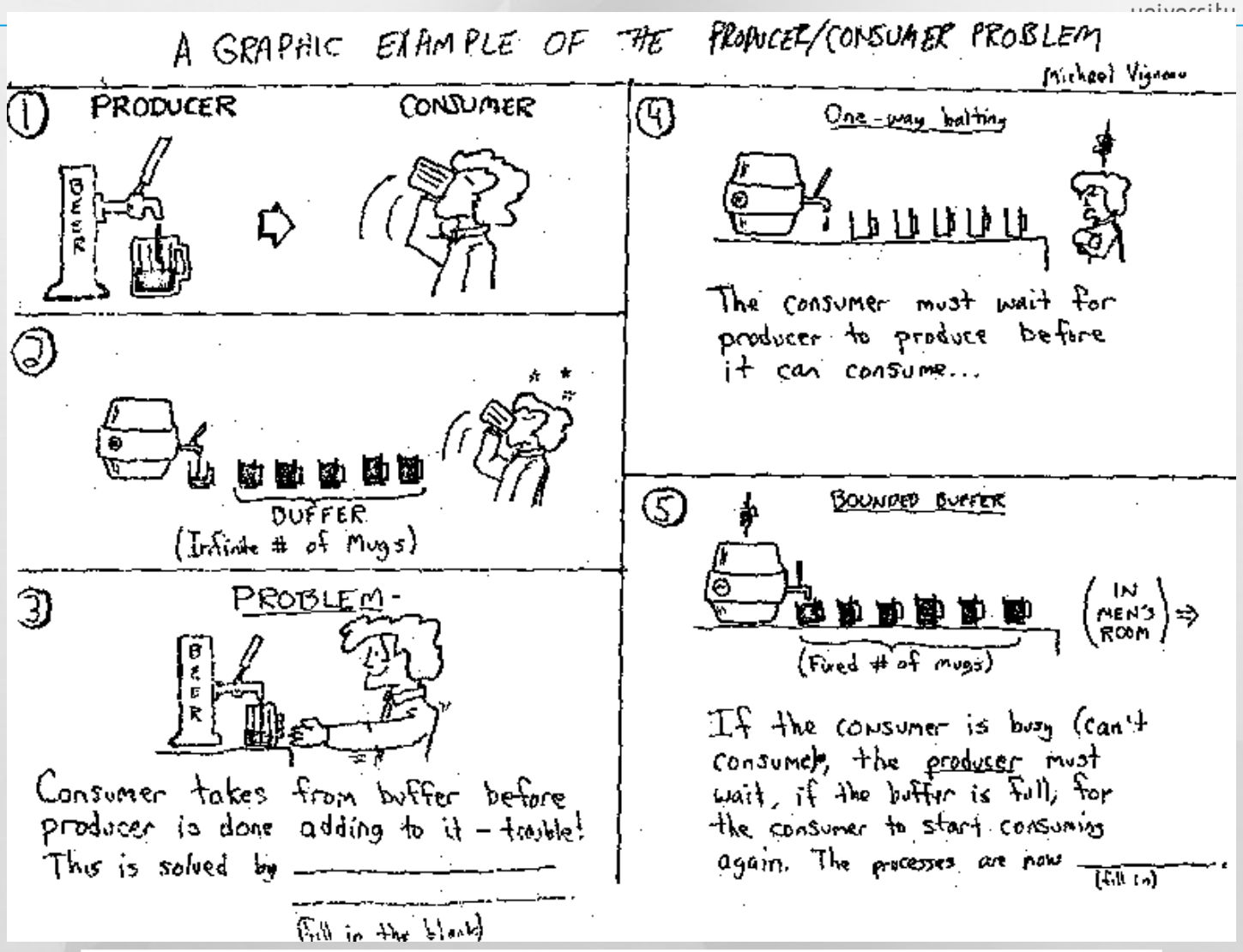


V Operation - Raise

- **Semaphore is incremented**
- **If a process waits**
  - the respective process gets unblocked and
  - The process continues in P operation with decrementing the Semaphore



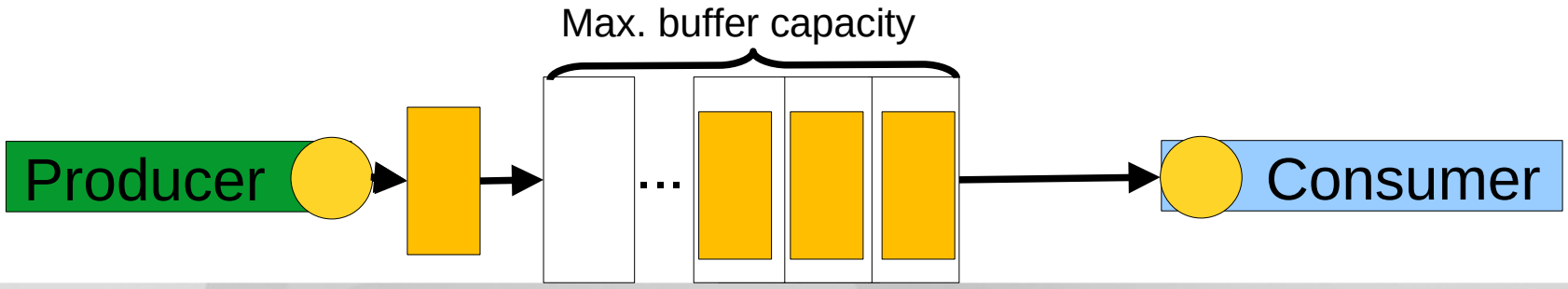
Producer – Consumer Problem



Source: http://www.ccs.neu.edu/home/kenb/tutorial/example.gif

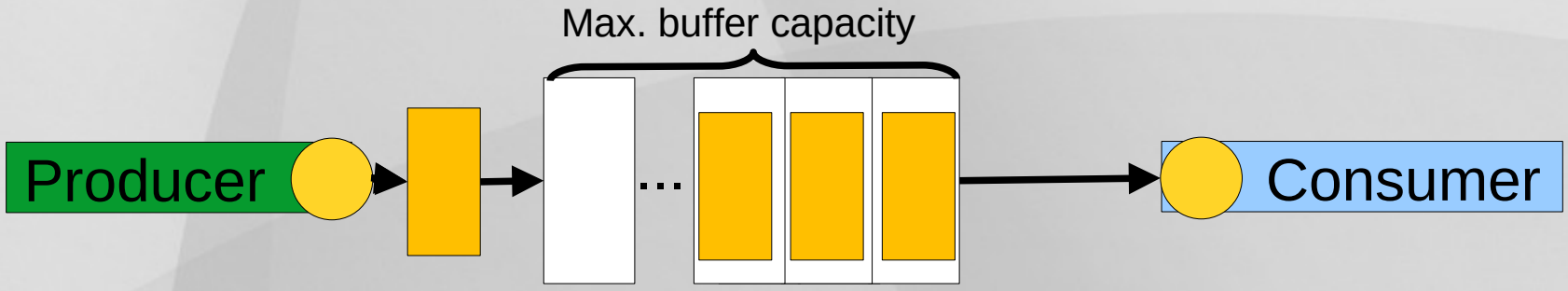
Producer – Consumer Example I

- **Producer sends data to a consumer**
- **Consumer receives data for further calculations or control**
- **Buffer with bounded capacity is used (typical remedy for data production asynchronous to data consumption)**
  - Minimizing waiting times for consumer
  - Providing space for data which is are not consumed in time
- **Mutual exclusion needed to avoid data inconsistencies**
- **Buffer full** → producer must be blocked
- **Buffer empty** → consumer must be *blocked (prevention from idling / busy waiting)*



Producer – Consumer Example II

- **Producer sends data to a consumer**
- **3 Semaphores**
  - **Empty:** counts empty spaces in the buffer
    - Value is increased by consumer (V operation)
    - Value is reduced by producer (P Operation)
      - If 0 → producer is blocked
  - **Filled:** counts allocated spaces in the buffer
    - Value is increased by producer (V operation)
    - Value is reduced by consumer (P Operation)
      - If 0 → consumer is blocked
  - **Mutex:** handles mutual exclusion access to the buffer + semaphores Filled & Empty



C-System Calls for Semaphores in Posix

TH Aschaffenburg
university of applied sciences

- **sem_init():** Create a new **unnamed** semaphore and thereby specify the initial value
- **sem_open():** Create a new **named** semaphore and thereby specify the initial value
- **sem_post():** Increment the value of a semaphore (V operation)
- **sem_wait():** Decrement the value of a semaphore (P operation). Blocking operation
- **sem_trywait():** Decrement the value of a semaphore (P operation). Non-blocking operation
- **sem_timedwait():** Decrement the value of a semaphore (P operation). Blocking operation but with a timeout
- **sem_getvalue():** Request the value of a semaphore
- **sem_destroy():** Erase an **unnamed** semaphore
- **sem_close():** Close a **named** semaphore
- **sem_unlink():** Erase a **named** semaphore

- Named POSIX semaphores in Linux: /dev/shm/sem.<name>

- **Note: Formal parameters are left out in this overview!**

Producer – Consumer C-Code Example with Posix API

TH Aschaffenburg
university of applied sciences

- **9_4_producer_consumer_example_posix_consumer**
- **9_4_producer_consumer_example_posix_producer**

- **$> ls -la /dev/shm/**
  pc_shm sem.empty_sem sem.full_sem sem.mutex_sem

- **Do not forget to remove ‚pc_shm‘ and semaphores by hand**

Mutex – Binary Semaphore

TH Aschaffenburg
university of applied sciences

- **Mutex**, compound from **mut**ual **ex**clusion, as a **0/1-semaphore**
- Protection of critical section, if only 1 single process is allowed at any given point in time

- Two states:
  - Lock, Occupied (eq. to value 1)
  - Unlock, Not occupied (eq. to value 0)

- Easier to implement, mostly for coordination of threads
  - pthread_mutex_init, pthread_mutex_unlock, pthread_mutex_lock, pthread_mutex_trylock, pthread_mutex_timedlock, pthread_mutex_destroy

Mutex Example in C for Posix Threads

TH Aschaffenburg
university of applied sciences

- **9_4_pthread_mutex_example_posix**
- **Here, we use 3 threads counting up NUM_ITER each!**

Monitor

TH Aschaffenburg
university of applied sciences

- **Further concept to secure critical sections**
- **Module with a data structure and access operations**
  - **Semaphores / mutexe are used as OS** internally
- **Easier to use, less error prone, compared to semaphore or mutex**
- **in Java**
  - *Synchronized, Wait, notify, notifyAll*
- **No monitor concept for Pascal, C, C++ or PHP**

Inter Process Communication

TH Aschaffenburg
university of applied sciences

- Semaphore✔
- Mutex ✔
- Monitor ✔